

MICROCONTROLLERS

Labo 1

Naslag

Gegenereerd op 31/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Programmeerconcepten

[Arrays](#)

Vermogen schakelen

[Vermogen schakelen](#)

Displays

[Het 7-segment display](#)

[Multiplexing](#)

Datasheets

[P2N2222A \(2N2222\) los document](#)

NASLAG

Arrays

Tot nu toe hield elke variabele één waarde bij. Zodra je met acht leds of tien cijferpatronen werkt, wordt dat onwerkbaar. Een array is één variabele die een hele rij waarden bevat, en die je met een lus kan doorlopen.

Ken je de lessen Technisch programmeren (C#), dan is een array te vergelijken met een **List**. Het grote verschil is dat je bij een array op voorhand vastlegt hoeveel plaatsen er zijn, en dat aantal verandert daarna niet meer.

Een array declareren

Je zet vierkante haakjes achter de naam en geeft de beginwaarden mee tussen accolades:

```
int lijst[] = {10, 4, -1, 0};
```

CPP

Dit is één variabele, `lijst`, met vier plaatsen erin. Alle plaatsen hebben hetzelfde type, hier `int`. Je kan dus geen getallen en tekst door elkaar in dezelfde array steken.

Een waarde opvragen of aanpassen

Elke plaats heeft een **index**, en die begint bij 0. In een array van vier plaatsen zijn de geldige indexen dus 0, 1, 2 en 3:

Index	0	1	2	3
Waarde	10	4	-1	0

De array `lijst` plaats per plaats

```
1 int eersteItem = lijst[0]; // 10, de eerste plaats
2 lijst[1] = 5;           // de tweede plaats krijgt de waarde 5
```

CPP

⚠ BUITEN DE ARRAY LEZEN MERKT NIEMAND OP

Schrijf je `lijst[4]` in een array van vier plaatsen, dan lees of schrijf je geheugen dat niet van jou is. De compiler zegt daar niets over en je sketch loopt gewoon door, maar je krijgt een willekeurige waarde terug of je overschrijft een andere variabele. Dat soort fout is achteraf lastig te vinden, dus tel je indexen goed na.

Het aantal plaatsen laten berekenen

Je kan het aantal plaatsen zelf tellen, maar dan moet je dat getal aanpassen telkens je er een waarde bijzet. Laat de compiler het liever uitrekenen:

```
int aantal = sizeof(lijst) / sizeof(lijst[0]);
```

CPP

`sizeof(lijst)` is hoeveel geheugen de hele array inneemt, `sizeof(lijst[0])` hoeveel één plaats inneemt. De deling geeft dus het aantal plaatsen. Voeg je later een waarde toe, dan klopt het getal nog altijd.

Een array doorlopen met een lus

Met een `for-lus` behandel je alle plaatsen zonder de code te herhalen:

```

1  const int ledPins[] = {3, 4, 5, 6};
2  const int aantalLeds = sizeof(ledPins) / sizeof(ledPins[0]);
3
4  void setup()
5  {
6      for (int i = 0; i < aantalLeds; i++)
7      {
8          pinMode(ledPins[i], OUTPUT);
9      }
10 }

```

De tellervariabele van de lus dient meteen als index. Vier pinnen instellen kost zo drie regels in plaats van vier, en bij zestien leds nog altijd drie.

Tweedimensionale arrays

Soms hoort bij elke plaats niet één waarde maar een hele rij. Denk aan een 7-segment display: voor elk cijfer van 0 tot 9 moet je weten welke van de zeven segmenten aan moeten. Dat zijn tien rijen van zeven waarden, en dan gebruik je **twee** paar haakjes:

```

1  // 3 rijen van 2 waarden
2  const int tabel[3][2] = {
3      {1, 2},
4      {3, 4},
5      {5, 6}
6  };

```

Het eerste getal tussen de haakjes is het aantal **rijen**, het tweede het aantal **kolommen**. Je spreekt een waarde aan met twee indexen, in diezelfde volgorde: eerst de rij, dan de kolom.

	kolom 0	kolom 1
rij 0	<code>tabel[0][0] = 1</code>	<code>tabel[0][1] = 2</code>
rij 1	<code>tabel[1][0] = 3</code>	<code>tabel[1][1] = 4</code>
rij 2	<code>tabel[2][0] = 5</code>	<code>tabel[2][1] = 6</code>

Welke waarde staat waar in `tabel`



BIJ EEN TWEEDEMENTIONALE ARRAY MOET JE HET AANTAL RIJEN WEL ZELF SCHRIJVEN

Bij een gewone array mag je de haakjes leeg laten (`int lijst[] = {...}`), want de compiler telt zelf. Bij twee dimensies moet minstens het aantal kolommen erin staan, anders weet hij niet waar een rij ophoudt. Schrijf ze in de praktijk gewoon allebei, dan lees je meteen hoe groot de tabel is.

Rijen en kolommen laten berekenen

Dezelfde `sizeof`-truc werkt ook bij twee dimensies, alleen moet je ze twee keer toepassen:

CPP

```
1 // De hele tabel gedeeld door één rij geeft het aantal rijen
2 const int aantalRijen = sizeof(tabel) / sizeof(tabel[0]);
3
4 // Eén rij gedeeld door één element geeft het aantal kolommen
5 const int aantalKolommen = sizeof(tabel[0]) / sizeof(tabel[0][0]);
```

`tabel[0]` is de eerste rij, en die is zelf een gewone array. Daarom werkt de bekende deling er ook op. Doe je dit, dan blijft je programma kloppen wanneer je later een rij toevoegt of de tabel breder maakt, zonder dat je ergens een getal moet aanpassen.

Een rij uit de tabel gebruiken

De gebruikelijke vorm is een lus over de kolommen, waarbij de rij bepaalt welk geval je behandelt:

CPP

```
1 // Per cijfer: welke van de 7 segmenten moeten aan? 1 = aan.
2 const bool cijferPatronen[10][7] = {
3     {1, 1, 1, 1, 1, 1, 0}, // 0
4     {0, 1, 1, 0, 0, 0, 0} // 1
5     // ... enzovoort tot en met 9
6 };
7
8 void toonCijfer(int cijfer)
9 {
10     for (int i = 0; i < 7; i++)
11     {
12         // rij = het cijfer, kolom = het segment
13         digitalWrite(segPins[i], cijferPatronen[cijfer][i]);
14     }
15 }
```

Zonder array zou je hier tien keer zeven `digitalWrite`-regels moeten schrijven, of een selectie met tien takken. Nu staat de kennis in een tabel die je in één oogopslag kan nalezen en aanpassen, en blijft de code die ze gebruikt drie regels lang.

Vermogen schakelen

Een uitgangspin van je Arduino levert ongeveer 20 mA, en dat is minder dan je vaak nodig hebt. Met een transistor laat je een kleine stroom uit die pin een veel grotere stroom schakelen, en die grote stroom komt dan ergens anders vandaan: een externe voeding of de 5V-pin.

Hoeveel stroom een pin levert

Een digitale uitgangspin van een Arduino is geen voeding. Hij is bedoeld om een signaal te geven, en de stroom die hij daarbij kan leveren is beperkt:

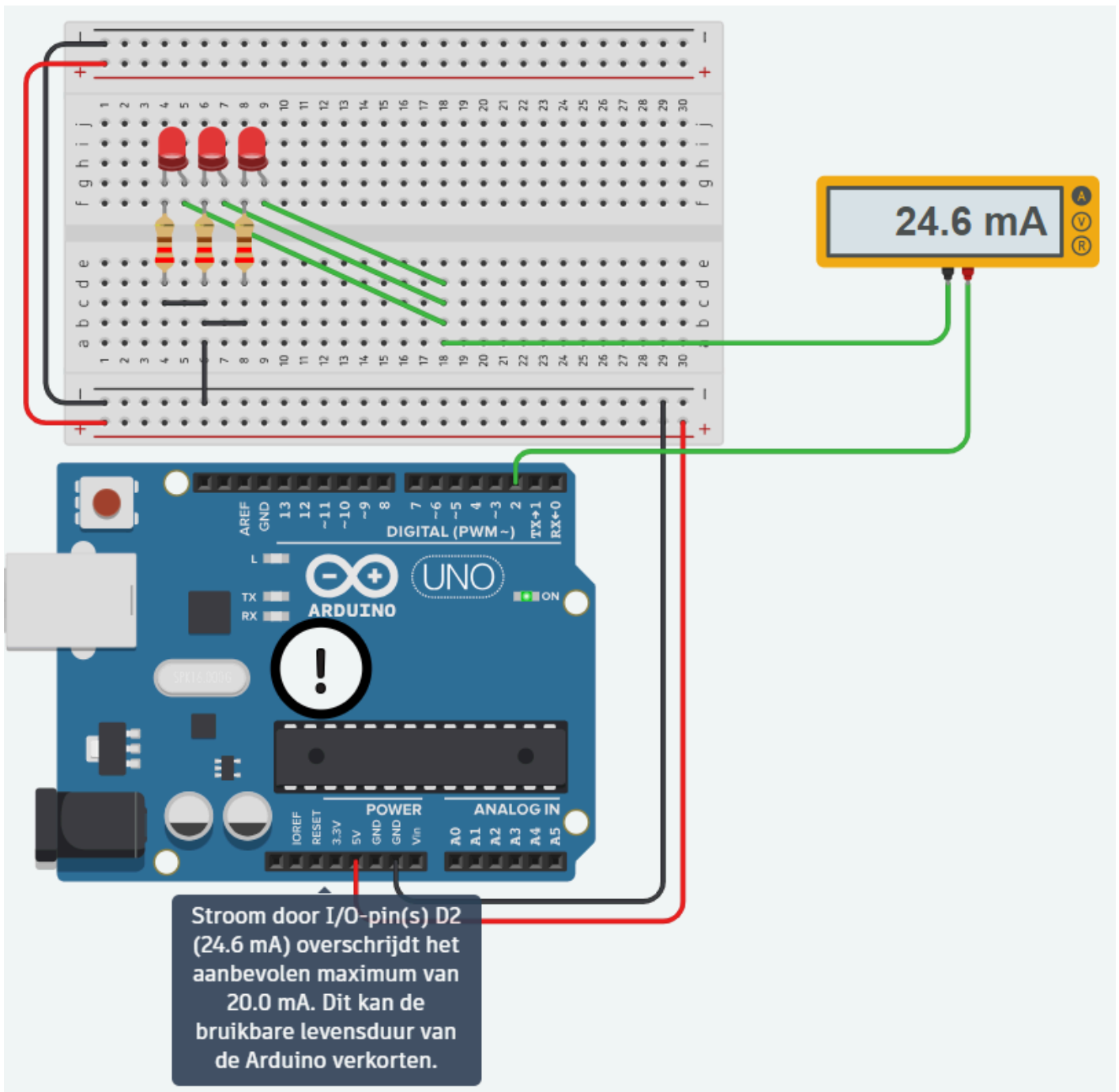
Grens	Waarde
Aangeraden per pin	20 mA
Absoluut maximum per pin	40 mA
Alle pinnen samen	200 mA

Hoeveel stroom je nodig hebt, weet je al. Bij [de wet van Ohm](#) rekende je de voorschakelweerstand van een led uit: 160 Ω in theorie, afgerond naar de 220 Ω die je echt hebt, en daardoor loopt er **14,5 mA** door één led in plaats van de nominale 20 mA.

Zet er drie naast elkaar op dezelfde pin, dan tel je die stromen op:

Aantal leds	Stroom door de pin	
1	14,5 mA	Onder de aangeraden 20 mA
2	29 mA	Boven de aangeraden 20 mA
3	43,5 mA	Boven het absolute maximum van 40 mA

Bij drie leds vraag je dus meer dan de pin ooit mag leveren. Kijk wat TinkerCAD daarvan maakt:



[Klik op de afbeelding om te vergroten](#)

De meter wijst geen 43,5 mA aan maar **24,6 mA**. Die 24,6 mA is niet wat je schakeling vraagt, het is wat de pin er nog van kan maken. Onder een belasting die hij niet aankan zakt zijn uitgangsspanning weg, en met minder spanning over dezelfde weerstanden loopt er minder stroom. Je leds branden dus zwakker dan waarvoor je ze berekend hebt.

Kapot gaat je pin er niet meteen van, maar de uitgangstrap van de microcontroller wordt warm, en hoe verder je boven de grens zit hoe korter je bord meegaat. Onthou vooral het getal dat je zelf berekende: je hebt **43,5 mA** nodig, en dat is wat je schakeling straks moet kunnen leveren.

DE 5V-PIN IS IETS ANDERS DAN EEN UITGANGSPIN

De 5V-pin op het bord komt rechtstreeks van de spanningsregelaar of van je USB-kabel, en niet uit de microcontroller. Die pin levert honderden milliampère. De oplossing bestaat er dus in de stroom voor je leds daar te halen, en de uitgangspin alleen nog te laten bepalen *of* die stroom loopt.

Een relais of een transistor

Er zijn twee componenten die een grote stroom kunnen schakelen met een klein signaal. Een **relais** is een schakelaar die je met een elektromagneet bedient: er zit een spoel in die een contact fysiek naar zich toe trekt. Een **transistor** doet hetzelfde zonder bewegende delen.

	Relais	Transistor
Prijs	Ongeveer een euro	Enkele centen
Schakeltijd	5 tot 10 ms	Microseconden
Slijtage	Bewegende contacten, dus een eindig aantal schakelingen	Geen bewegende delen
Geluid	Hoorbare klik bij elke schakeling	Stil
Wat de spoel of de basis zelf trekt	De spoel van een 5V-relais trekt zo'n 70 mA	Een fractie van de stroom die je schakelt
Dimmen met PWM	Onmogelijk, het contact staat open of dicht	Mogelijk, zie analogWrite() uit labo 2

Kijk vooral naar de voorlaatste rij. Een relaisspoel trekt zelf 70 mA, en dat is opnieuw meer dan de 20 mA van je pin. Om een relais te schakelen heb je dus sowieso een transistor nodig, en dan kan je die transistor net zo goed rechtstreeks op je leds zetten.

Waar een relais wel op zijn plaats is: bij netspanning, want het contact is elektrisch volledig gescheiden van je Arduino, en bij schakelingen waar de stroom beide kanten op moet kunnen. Voor de gelijkstroom van dit labo neem je een transistor.

i ER BESTAAT NOG EEN DERDE COMPONENT

De **MOSFET**, een veldeffecttransistor of kortweg FET, doet hetzelfde werk. Het verschil zit in hoe je hem stuurt: een gewone transistor stuur je met een *stroom* in zijn basis, een MOSFET met een *spanning* op zijn gate. Er loopt daar vrijwel geen stroom, dus bij een MOSFET valt er ook geen stuurweerstand te berekenen.

Daar staat één ding tegenover. Een gewone MOSFET gaat pas voluit open bij ongeveer 10 V op zijn gate, en jouw pin geeft er 5. Je hebt dus een type nodig dat als **logic level** verkocht wordt. Neem je een gewone, dan staat hij half open en wordt hij warm, terwijl je schakeling ondertussen lijkt te werken.

De drie aansluitingen

Een NPN-transistor heeft drie aansluitingen. Stuur je een kleine stroom in de **basis**, dan laat hij een veel grotere stroom door tussen **collector** en **emitter**.

Aansluiting	Waar ze naartoe gaat
B (basis)	Via een weerstand naar de uitgangspin van de Arduino
C (collector)	Naar wat je wil schakelen, hier de leds
E (emitter)	Naar GND

Welk pootje welke aansluiting is, staat op de eerste pagina van de [datasheet](#). Rechtsboven vind je het schemasymbool met de drie namen erbij, en daaronder de TO-92 behuizing met de pootjes genummerd 1, 2 en 3 in dezelfde volgorde.

P2N2222A

Amplifier Transistors

NPN Silicon

Features

- These are Pb-Free Devices*

MAXIMUM RATINGS ($T_A = 25^\circ\text{C}$ unless otherwise noted)

Characteristic	Symbol	Value	Unit
Collector – Emitter Voltage	V_{CE0}	40	Vdc
Collector – Base Voltage	V_{CB0}	75	Vdc
Emitter – Base Voltage	V_{EB0}	6.0	Vdc
Collector Current – Continuous	I_C	600	mA dc
Total Device Dissipation @ $T_A = 25^\circ\text{C}$ Derate above 25°C	P_D	625 5.0	mW mW/ $^\circ\text{C}$
Total Device Dissipation @ $T_C = 25^\circ\text{C}$ Derate above 25°C	P_D	1.5 12	W mW/ $^\circ\text{C}$
Operating and Storage Junction Temperature Range	T_J, T_{stg}	-55 to +150	$^\circ\text{C}$

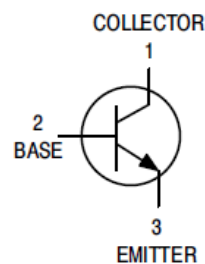
THERMAL CHARACTERISTICS

Characteristic	Symbol	Max	Unit
Thermal Resistance, Junction to Ambient	$R_{\theta JA}$	200	$^\circ\text{C/W}$

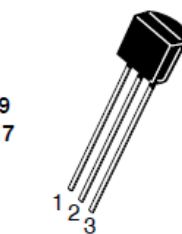


ON Semiconductor®

<http://onsemi.com>



TO-92
CASE 29
STYLE 17



STRAIGHT LEAD
BULK PACK

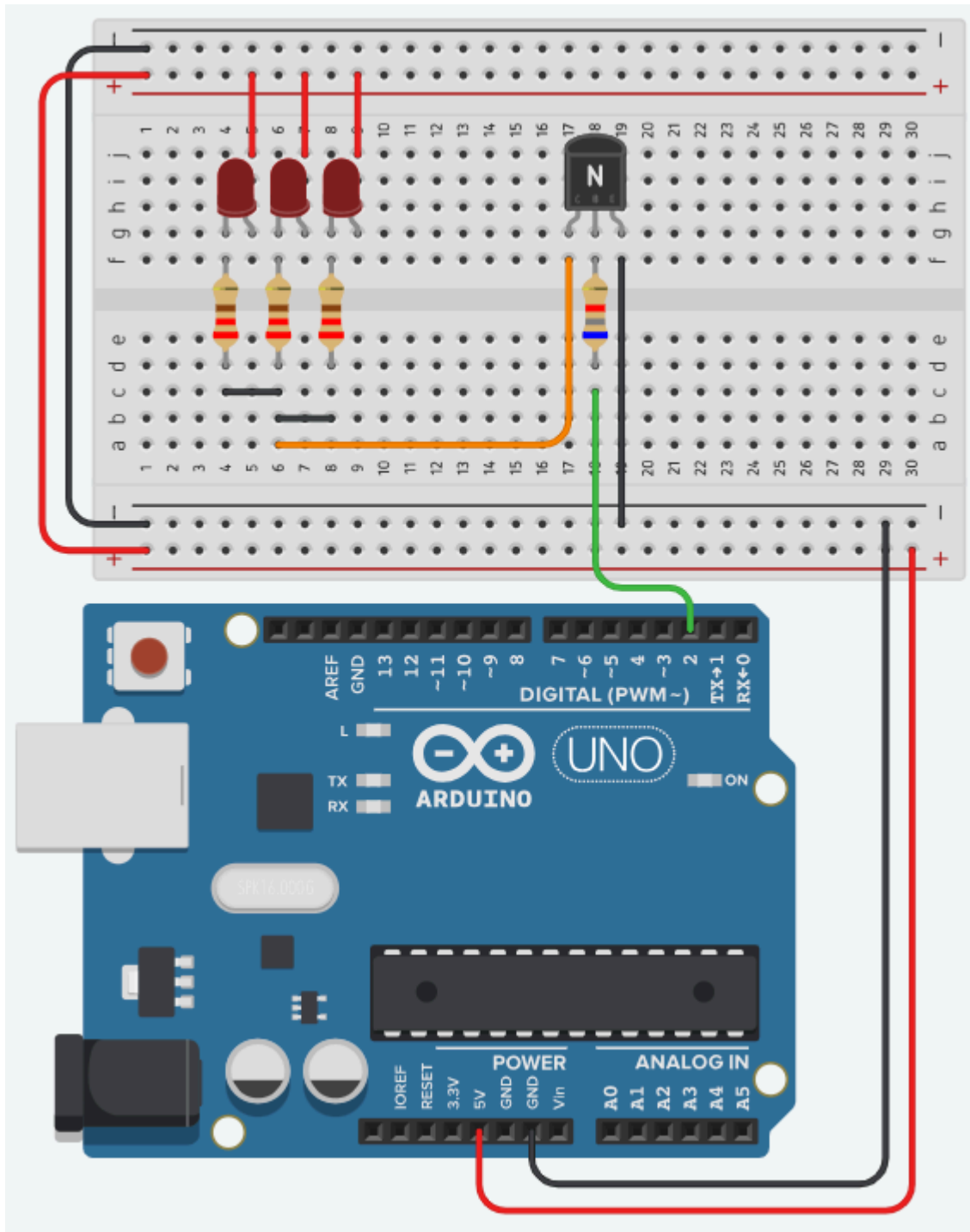


BENT LEAD
TAPE & REEL
AMMO PACK

Klik op de afbeelding om te vergroten

Diezelfde pagina geeft je meteen de grenzen van het onderdeel. De collectorstroom mag oplopen tot **600 mA** en tussen collector en emitter mag 40 V staan. Een 2N2222 kan dus veel meer schakelen dan wat je er in dit labo mee doet.

Zo ziet de schakeling van de drie leds er dan uit:



[Klik op de afbeelding om te vergroten](#)

De leds hangen **boven** de transistor: van de plus van de voeding, door de leds en hun voorschakelweerstand, naar de collector. De transistor zit tussen de leds en de massa en laat de stroom er al dan niet door. Naar de Arduino lopen er drie draden: de 5V en de massa voor de voeding van de leds, en pin 2 naar de basisweerstand. In TinkerCAD is de **2N2222** gemodelleerd, en die gebruik je hier.

Je gebruikt de transistor op deze manier niet om een signaal te versterken, maar als een **schakelaar die je met software bedient**. Hij staat ofwel helemaal open, ofwel helemaal dicht, en niets ertussen. Die

volledig open toestand heet **satueratie**. Ze is wat je wil, want dan staat er nauwelijks spanning over de transistor en maakt hij dus nauwelijks warmte.

De basisweerstand berekenen

Tussen je Arduino-pin en de basis hoort een weerstand. Zonder die weerstand loopt er een ongelimiteerde stroom je basis in en gaat je pin, je transistor of allebei eraan.

De formule voor die weerstand is de [wet van Ohm](#) toegepast op de basiskring:

$$R_b = \frac{V_{cc} - V_{be}}{I_b}$$

Je hebt dus drie dingen nodig. Hieronder staat het voorbeeld van de drie leds helemaal uitgewerkt. In de oefening van dit labo doe je dezelfde berekening voor een 7-segment display, met een andere stroom en dus een andere weerstand.

1. V_{cc} , de spanning op je pin

Een uitgangspin van de UNO die hoog staat geeft 5 V. Dus $V_{cc} = 5$ V.

2. V_{be} , de spanning over de basis-emitterjunctie

Ook die staat als grafiek in de [datasheet](#). Zoek weer je collectorstroom van 43,5 mA op, ga omhoog tot de curve van $V_{BE(sat)}$ en lees links af: ongeveer **0,7 V**. Bij zowat elke siliciumtransistor kom je op die waarde uit.

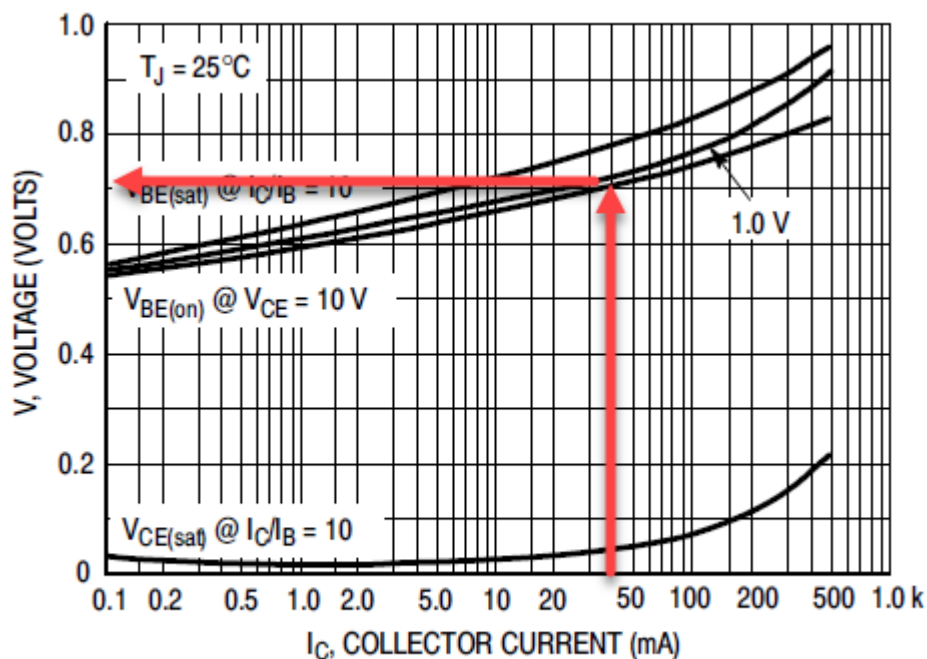


Figure 11. "On" Voltages

[Klik op de afbeelding om te vergroten](#)

Onderaan diezelfde grafiek loopt nog een curve, $V_{CE(sat)}$. Dat is de spanning die de transistor zelf overhoudt wanneer hij volledig openstaat. Bij 43,5 mA is dat maar een paar honderdsten van een volt, en dat is precies de bedoeling: wat de transistor niet zelf opneemt, komt bij je leds terecht.

3. I_b , de stroom die je in de basis moet sturen

Die volgt uit de stroom die je wil schakelen, gedeeld door de stroomversterkingsfactor h_{FE} van de transistor:

$$I_b = \frac{I_c}{h_{FE}}$$

I_c is de collectorstroom, en die heb je hierboven al uitgerekend: drie leds van 14,5 mA, dus **43,5 mA**. Merk op dat je hier vertrekt van wat je schakeling *nodig heeft*, en niet van de 24,6 mA die de overbelaste pin nog kon leveren. De h_{FE} zoek je weer op in de datasheet.

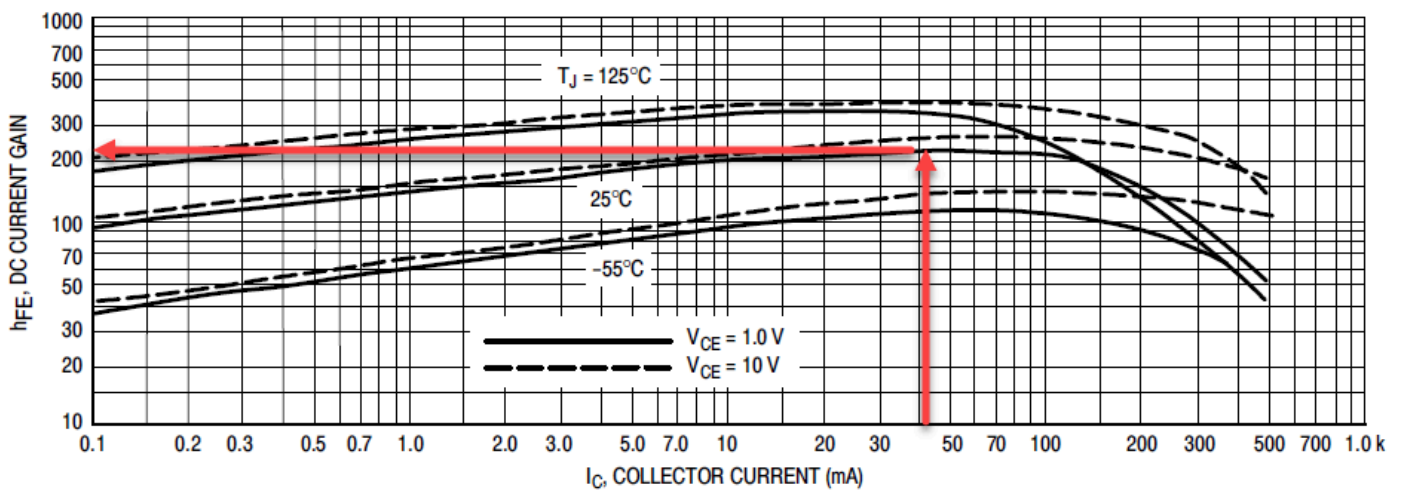


Figure 3. DC Current Gain

[Klik op de afbeelding om te vergroten](#)

Het is een grafiek en geen enkel getal, dus je moet ze aflezen bij jouw collectorstroom. Zoek 43,5 mA op de horizontale as, ga omhoog tot de curve van 25°C , en lees links af: een h_{FE} van ongeveer 200.

⚠ NEEM RUIM MARGE OP DE HFE

Reken je met die 200, dan stuur je precies genoeg basisstroom om je 43,5 mA te halen *als alles meezit*, en dat is te krap. Kijk nog eens naar de grafiek: bij dezelfde 43,5 mA leest de curve van -55 °C ongeveer 100 af en die van 125 °C ruim 300. Alleen de temperatuur laat je hFE dus al een factor drie schommelen, en daar komt de spreiding van exemplaar tot exemplaar nog bovenop. Rechts op de grafiek zie je bovendien dat de hFE vanaf een paar honderd milliampère instort.

Bovendien wil je dat de transistor in saturatie gaat en dus helemaal openstaat. Een transistor die maar half openstaat verstookt het verschil in warmte.

Deel de hFE daarom ruim door. Hier deel je 200 door 5 en reken je verder met **hFE = 40**. Dat ligt onder de honderd van de koudste curve, dus je zit goed over het hele temperatuurbereik. Je stuurt dan vijf keer meer basisstroom dan strikt nodig, en dat is de bedoeling.

De berekening

Met hFE = 40 wordt de basisstroom:

$$I_b = \frac{0,0435 \text{ A}}{40} = 0,00109 \text{ A} = 1,09 \text{ mA}$$

En daarmee de basisweerstand:

$$R_b = \frac{5 \text{ V} - 0,7 \text{ V}}{0,00109 \text{ A}} = 3954 \Omega$$

Die waarde bestaat niet als standaardweerstand. Je kiest de dichtstbijzijnde die je hebt liggen, en dat is **3,9 kΩ**.

i DEZE KEER ROND JE NAAR BENEDEN AF

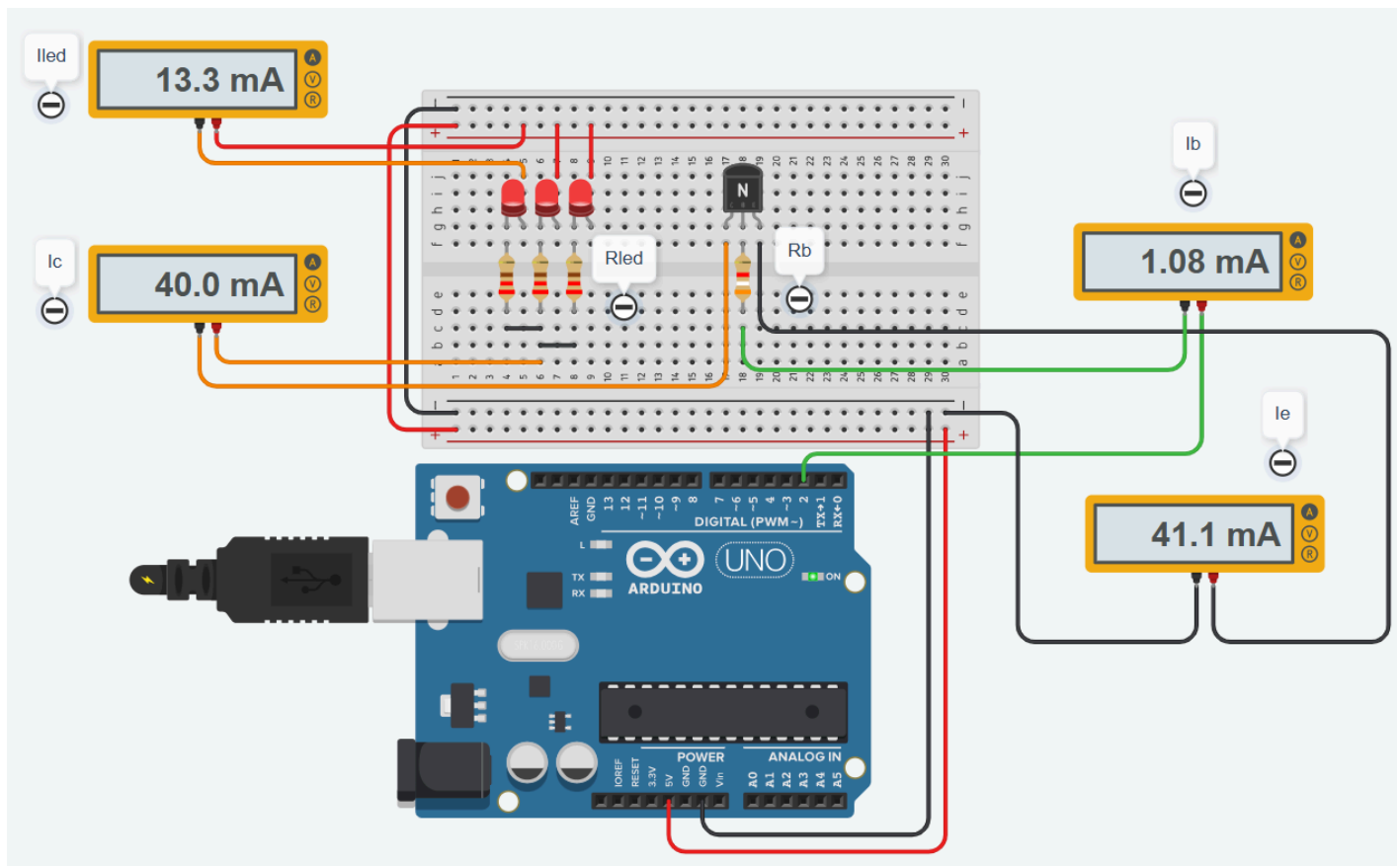
Bij de voorschakelweerstand van je led rondde je 160 Ω naar boven af, naar 220 Ω. Hier rond je 3954 Ω naar beneden af, naar 3900 Ω. Dat lijkt tegenstrijdig, maar het is twee keer dezelfde vraag: **welke kant is veilig voor dit onderdeel?**

Een grotere weerstand geeft minder stroom. Voor een led is dat de veilige kant, want te veel stroom maakt hem stuk. Voor een transistorbasis is het net de verkeerde, want te weinig basisstroom laat de transistor half openstaan en dan wordt hij warm. Te veel basisstroom is zelden een probleem, te weinig wel.

Kijk nog eens naar wat je pin nu doet. In plaats van 43,5 mA te moeten leveren, stuurt hij nog 1,09 mA de basis in. Dat is veertig keer minder, en ruim onder de 20 mA die hij aankan. De stroom voor de leds komt uit de 5V-pin en loopt via de collector en de emitter naar de massa. De pin bepaalt alleen nog of dat gebeurt.

De schakeling doormeten

Bouw je deze schakeling in TinkerCAD met 220 Ω per led en de berekende 3,9 k Ω in de basis, en hang je er op de vier interessante plaatsen een ampèremeter in, dan zie je dit:



[Klik op de afbeelding om te vergroten](#)

Meting	Waar de stroom doorheen loopt	Gemeten	Berekend
I_{led}	Door één led	13,3 mA	14,5 mA
I_c	In de collector, dus de drie leds samen	40,0 mA	43,5 mA
I_b	In de basis, dus wat je pin levert	1,08 mA	1,09 mA
I_e	In de emitter	41,1 mA	

De basisstroom komt op een honderdste milliampère uit waar je berekening hem voorspelde. De emitter heeft geen eigen berekening nodig, want zijn waarde volgt uit de twee andere:

$$I_e = I_b + I_c = 1,08 + 40,0 = 41,1 \text{ mA}$$

Alles wat er langs de basis en langs de collector binnenkomt, verlaat de transistor langs de emitter. Dat is meteen een goede controle op je schakeling: klopt die optelling niet, dan meet je ergens verkeerd of loopt er stroom waar je ze niet verwacht.

i DE GEMETEN COLLECTORSTROOM LIGT ONDER DE BEREKENDE

Dat verschil kan je narekenen. Over de weerstand staat $0,0133 \times 220 = 2,93 \text{ V}$, dus de led en de transistor samen nemen $5 - 2,93 = \mathbf{2,07 \text{ V}}$ voor hun rekening. In je berekening ging je uit van $1,8 \text{ V}$ voor de led en niets voor de transistor.

Waar zit die $0,27 \text{ V}$ dan? Aan de transistor ligt het nauwelijks: de $V_{CE(sat)}$ -curve hierboven geeft bij deze stroom maar een paar honderdsten van een volt. Het is dus vooral de led. Op [de wet van Ohm](#) staat dat de $V_{forward}$ van een led meestal tussen $1,8 \text{ V}$ en 2 V ligt, en jij nam de onderkant van dat bereik. Deze led zit aan de bovenkant.

Voor het dimensioneren maakt dat niets uit, want het valt volledig binnen de marge van factor 5 op de h_{FE} . Reken gerust verder met $1,8 \text{ V}$.

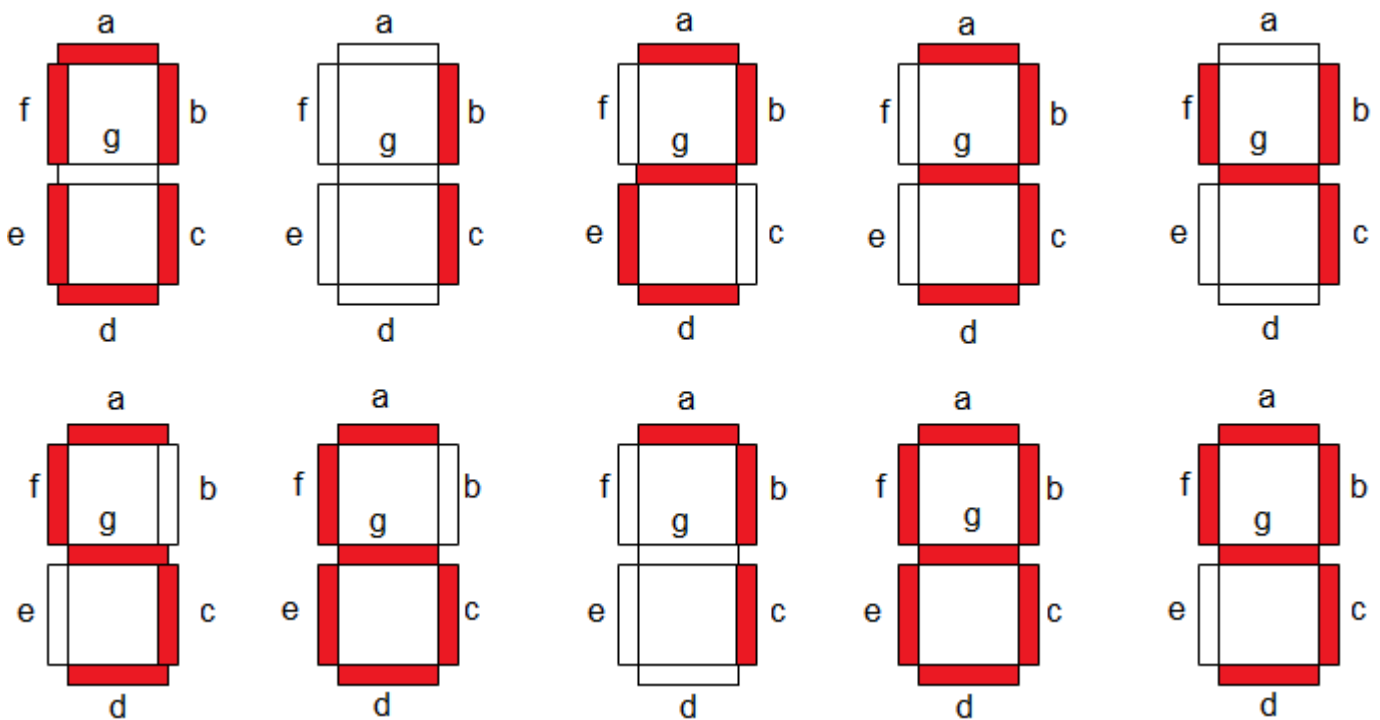
Het 7-segment display

Een 7-segment display is zeven leds in de vorm van een acht, plus meestal een achtste voor de decimale punt. Elk cijfer dat je toont, is een keuze welke van die zeven branden.

Drie woorden komen in dit labo voortdurend terug. Een **display** is één zo'n onderdeel met zeven segmenten. Een **segment** is één van die leds. Een **cijfer** is de waarde 0 tot en met 9 die je op een display zet. Een dubbel display bevat er twee naast elkaar, het linkse en het rechtse.

De segmenten en hun namen

De zeven segmenten heten a tot en met g, en die namen zijn een afspraak die overal hetzelfde is. Segment a is de bovenste balk, en vanaf daar loop je met de klok mee: b rechtsboven, c rechtsonder, d onderaan, e linksonder, f linksboven, en g de balk in het midden. De decimale punt heet **dp**.



[Klik op de afbeelding om te vergroten](#)

Elk segment heeft zijn eigen aansluiting, en daarnaast is er één **gemeenschappelijke** pin waar alle zeven leds met hun andere kant op uitkomen. Een display sturen kost je dus zeven Arduino-pinnen, acht als je ook dp nodig hebt.

Common anode of common cathode

Er bestaan twee soorten, en welke je hebt bepaalt of je een segment met **HIGH** of met **LOW** laat branden. Het verschil zit in wat er met die gemeenschappelijke pin gebeurt:

	Common cathode	Common anode
De gemeenschappelijke pin gaat naar	GND	+5 V
Een segment brandt bij	HIGH	LOW
De Arduino-pin	sourcet de stroom	sinkt de stroom

De twee soorten 7-segment displays

Het is hetzelfde onderscheid als in [Sourcen en sinken](#).

⚠ ELK SEGMENT HEEFT ZIJN EIGEN WEERSTAND NODIG

De zeven segmenten delen hun gemeenschappelijke pin, maar niet hun stroom. Zet je één weerstand in die gemeenschappelijke tak, dan verdeelt de stroom zich over de segmenten die toevallig aan staan en verandert de helderheid met elk cijfer: een 1 (twee segmenten) brandt dan feller dan een 8 (zeven segmenten). Geef daarom elk segment zijn eigen serieweerstand. Een segment is een gewone led dus de berekening van de voorschakelweerstand is gelijkaardig aan die van een gewone led. Hoe je die precies berekent, staat op [De wet van Ohm](#).

Van cijfer naar patroon

Om een cijfer te tonen moet je weten welke segmenten erbij horen. Voor een 7 zijn dat a, b en c; voor een 0 alles behalve g. Die kennis leg je vast in een [tweedimensionale array](#): één rij per cijfer, één kolom per segment.

Cijfer	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1

De eerste vier cijfers, met 1 voor "segment aan"

Meerdere displays

Wil je twee cijfers tonen, dan heb je twee displays en minstens veertien pinnen nodig. Dat begint al lastig te worden voor een Arduino Uno of Leonardo want er zijn niet genoeg pinnen beschikbaar. Er zijn drie oplossingen: de eerste is [Multiplexing](#) en de andere (schuifregister en I2C I/O expander) bespreken we in labo 3 en 4.

Multiplexing

Twee cijfers tonen terwijl je maar zeven segmentpinnen gebruikt. Je stuurt de twee displays om beurten aan, en je wisselt zo snel dat je oog het verschil niet ziet.

Het probleem

Een **7-segment display** kost je zeven pinnen. Twee displays zouden er dus veertien kosten, en zoveel vrije pinnen heeft een Arduino UNO niet als er ook nog knoppen en leds aan moeten.

Een dubbel display lost dat hardwarematig half op: de zeven segmentaansluitingen van beide displays zitten aan elkaar geknoopt. Segment a van het linkse display en segment a van het rechtse hangen aan dezelfde pin. Alleen de **gemeenschappelijke** pin van elk display is apart uitgebracht.

Dat scheelt pinnen, maar het levert meteen een nieuw probleem op: zet je een patroon op de segmentpinnen, dan krijgen *beide* displays datzelfde patroon. Je kan dus onmogelijk tegelijk een 4 links en een 7 rechts tonen. Wat je wel kan, is kiezen welk display op dat moment stroom krijgt.

Te snel om te zien

Je toont de twee cijfers dus om beurten:

1. Zet het patroon van het **linkse** cijfer op de segmentpinnen.
2. Activeer alleen het linkse display. Even wachten.
3. Zet het patroon van het **rechtse** cijfer op de segmentpinnen.
4. Activeer alleen het rechtse display. Even wachten.
5. Opnieuw vanaf stap 1.

Doe je dat traag, dan zie je de twee cijfers om beurten flinkeren. Doe je het snel genoeg, dan kan je oog de twee niet meer uit elkaar houden en zie je twee cijfers naast elkaar staan. Dat heet **persistentie van het netvlies**: je oog blijft een lichtpunt nog een fractie van een seconde zien nadat het al gedoofd is. Vanaf ongeveer 50 wisselingen per seconde per display valt er niets meer te merken.

DIT IS DEZELFDE TRUC ALS BIJ FILM

Een film is een reeks stilstaande beelden die snel genoeg na elkaar komen om als beweging over te komen. Bij multiplexing gebeurt hetzelfde, alleen wissel je tussen twee displays in plaats van tussen beelden.

In code

Elk display heeft zijn eigen pin die bepaalt of het aan de beurt is. Die pin stuurt de basis van een transistor, want de gemeenschappelijke aansluiting van een display voert meer stroom af dan een uitgangspin mag hebben. Waarom dat zo is en hoe je die transistor dimensioneert, staat op [Vermogen schakelen](#).

Eén doorloop van de wissel ziet er dan zo uit. Je zet eerst het patroon op de gedeelde segmentpinnen en activeert daarna het display waar dat patroon bij hoort:

CPP

```
1  const int displayPins[2] = {9, 10}; // naar de basis van elke transistor
2
3  void kiesDisplay(int display)
4  {
5      for (int i = 0; i < 2; i++)
6      {
7          digitalWrite(displayPins[i], i == display ? HIGH : LOW);
8      }
9  }
10
11 void multiplexStap(int linksCijfer, int rechtsCijfer)
12 {
13     toonCijfer(linksCijfer);
14     kiesDisplay(0);
15     delay(5);
16
17     toonCijfer(rechtsCijfer);
18     kiesDisplay(1);
19     delay(5);
20 }
```

Met twee keer 5 ms duurt één volledige wissel 10 ms, dus elk display komt honderd keer per seconde aan de beurt, ruim genoeg.

⚠ ER MAG NIETS ANDERS LANG DUREN

Zolang je programma iets anders doet, staat er geen enkel display aan. Zet je ergens een `delay(1000)` om je teller elke seconde op te hogen, dan blijft je display die hele seconde donker. Gebruik daarvoor de `millis()`-aanpak: je kijkt telkens of er al genoeg tijd voorbij is, en ondertussen blijft de multiplexing doorlopen.

```
1  void loop()
2  {
3      multiplexStap(teller / 10, teller % 10);
4
5      if (millis() - vorigeTick >= 1000)
6      {
7          vorigeTick = millis();
8          teller = (teller + 1) % 100;
9      }
10 }
```

CPP

💡 HET LIJKT DONKERDER, EN DAT KLOPT

Elk display staat maar de helft van de tijd aan, dus het brandt minder fel dan wanneer je het rechtstreeks zou aansturen. Bij twee displays valt dat nauwelijks op. Ga je later multiplexen over vier of acht displays, dan wordt het wel merkbaar.